



**WYDZIAŁ FIZYKI
i INFORMATYKI STOSOWANEJ**
Uniwersytet Łódzki

Maciej Bujalski

Kierunek: informatyka

Specjalność: informatyka stosowana

Specjalizacja: aplikacje mobilne

Numer albumu: 386012

Rotacyjnie inwariantne sieci neuronowe

Praca magisterska

wykonana pod kierunkiem

dr Krzysztofa Podlaskiego

w Katedrze Systemów Inteligentnych, WFiIS UŁ

Łódź 2025

Spis treści

Wstęp	3
Cel i motywacja pracy	3
Opis pracy	4
Zakres tematyczny	5
Organizacja pracy	5
Podstawy teoretyczne	5
Wprowadzenie do sieci konwolucyjnych (CNN)	5
Inwariancja translacyjna i rotacyjna	5
Problemy z rotacyjną inwariancją w klasycznych CNN	5
Przegląd literatury (np. E(2)-Equivariant CNNs, CyCNN)	5
Opis zbiorów danych	5
MNIST (cyfry odręczne)	5
GTSRB Gray (znaki drogowe w odcieniach szarości)	5
GTSRB RGB (znaki drogowe)	5
LEGO (obiekty 3d rzutowane na 2d)	5
Sposób augmentacji danych: zakresy rotacji, łączenie zbiorów	5
Architektury modeli	5
Standardowe CNN	5
Rotacyjnie inwariantne sieci (np. CyResNet, CyVGG, G-CNN)	5
Transformacje polarne: linearpolar vs logpolar	5
Implementacja i środowisko eksperymentalne	5
Python, PyTorch	5
Struktura projektu	5
Automatyzacja: skrypty trenowania, testowania, ewaluacji	5
Obsługa GPU, Docker, WSL	5
Organizacja logów, modeli, confusion matrixów	5
Eksperymenty	6
Scenariusze trenowania/testowania (opis JSON)	6
Pomiar skuteczności (accuracy, macierze pomyłek)	6
Śledzenie metryk: średnia, mediana, odchylenie standardowe	6
Analiza skuteczności względem rotacji	6
Ranking modeli	6
Porównanie wyników	6
CyCNN vs klasyczne CNN	6
VGG vs CyVGG	6
Resnet vs CyResNet	6
CyVGG vs CyResNet	6
Wpływ transformacji (linearpolar vs logpolar)	6

Wydajność na różnych zbiorach	6
Wnioski	6
Skuteczność rotacyjnych architektur	6
Wnioski z automatyzacji i systematyzacji ewaluacji	6
Propozycje dalszych badań	6
Bibliografia	6
Aneks	6
Listingi kodów	6
Dodatkowe wykresy, tablice wyników	6

Wstęp

Obrazy otaczają nas z każdej strony: od zdjęć ze smartfonów, przez monitoring miejski, katalogi produktów i systemy kontroli jakości na liniach produkcyjnych, po systemy wspomagania jazdy. Choć współczesne modele rozpoznawania obrazu radzą sobie bardzo dobrze, w praktyce bywają wrażliwe na pozornie drobne zmiany takie jak obrócenie obiektu o kilkanaście stopni czy niewielki przechył kamery. To, co dla człowieka jest naturalne i natychmiast rozpoznawalne (znak drogowy pod kątem, cyfra obrócona na kartce), dla klasycznej konwolucyjnej sieci neuronowej bywa problemem. Rdzeń trudności to brak naturalnej inwariantności względem rotacji: standardowe CNN-y „z definicji” lepiej radzą sobie z przesunięciami niż z obrotami.

W ostatnich latach pojawiło się kilka dróg domknięcia tej luki. Jedną to poszerzanie danych o zrotowane przykłady - poprawia odporność, ale wydłuża trening i nie gwarantuje uogólnienia na wszystkie kąty. Druga to architektury z wbudowaną geometrią: sieci grupowo równoważne (G-CNN, E(2)-equivariant), sieci cykliczne operujące na wielu orientacjach oraz przekształcenia do układów polarnych (linear-polar i log-polar), które „prostują” rotacje do przesunięć. Cel jest wspólny: by model rozpoznawał „to samo” niezależnie od orientacji, bez agresywnego dublowania danych.

Niniejsza praca skupia się na praktycznej weryfikacji tych podejść. Przygotowano zbiory obejmujące m.in. odręczne litery/cyfry, znaki drogowe (w kolorze i w odcieniach szarości) oraz syntetyczne obiekty 3D rzutowane na 2D (np. klocki LEGO), a następnie rozszerzono je o kontrolowane rotacje. Zaimplementowano i porównano wybrane architektury rotacyjnie inwariantne i ich warianty bazowe w **PyTorchu**, mierząc wpływ transformacji (linear-polar vs. log-polar), wyboru architektury i zakresu kątów na jakość predykcji. Obliczenia realizowano na kartach **NVIDIA GeForce RTX 3060 12 GB**, co skróciło czas trenowania i umożliwiło szeroki przegląd eksperymentów; środowisko uruchomieniowe ustandaryzowano z użyciem **Dockera** dla powtarzalności.

Celem pracy jest nie tylko pokazanie, że „da się” uzyskać odporność na rotacje, ale przede wszystkim wskazanie, **kiedy i jakim kosztem** ją osiągamy oraz które techniki przynoszą największy zysk względem klasycznych CNN-ów, jak wpływają na stabilność i szybkość uczenia, a także które konfiguracje są najpraktyczniejsze w realnych zastosowaniach (OCR, rozpoznawanie znaków, analiza obiektów technicznych). W dalszej części pracy przedstawiono podstawy, dane i augmentację, architektury, środowisko eksperymentalne, protokoły ewaluacji oraz wyniki z analizą i wnioskami.

Cel i motywacja pracy

Konwolucyjne sieci neuronowe (CNN) charakteryzują się zdolnością do analizy obrazów z zachowaniem niezmienniczości względem translacji. Jednak wciąż brakuje powszechnie uznanych architektur, które zapewniłyby inwariantność względem rotacji. Celem niniejszej pracy jest analiza skuteczności nowych rozwiązań zaproponowanych w literaturze, ukierunkowanych na zapewnienie odporności modeli na rotację danych wejściowych (np. <https://arxiv.org/pdf/2007.10588.pdf>). W ramach pracy zostały przygotowane wzbogacone zbiory danych, obejmujące m.in. zdjęcia odręcznie napisanych liter, znaków drogowych (zarówno w kolorze, jak i w odcieniach szarości) oraz obiektów 3D rzutowanych na przestrzeń 2D (klocki LEGO), rozszerzone o różnorodne rotacje obrazów.

Na podstawie tych zbiorów danych przeprowadzono implementację i ewaluację wybranych archi-

tektur rotacyjnie inwariantnych z wykorzystaniem biblioteki PyTorch. Obliczenia zostały przeprowadzone przy użyciu kart graficznych NVIDIA GeForce RTX 3060 12GB, umożliwiających przyspieszenie procesów trenowania modeli. Otrzymane wyniki zostały porównane z rezultatami klasycznych sieci konwolucyjnych, w celu oceny realnych korzyści wynikających z zastosowania rozwiązań inwariantnych względem rotacji.

Opis pracy

Praca magisterska wykorzystuje zaawansowane technologie i narzędzia wspierające badania nad rotacyjnie inwariantnymi sieciami neuronowymi oraz ich zastosowaniem w przetwarzaniu obrazów. W realizacji projektu zastosowano następujące rozwiązania technologiczne:

- Język programowania Python – podstawowe narzędzie do implementacji algorytmów oraz obsługi frameworków uczenia maszynowego, dzięki swojej wszechstronności i bogatemu ekosystemowi bibliotek, takich jak PyTorch i TensorFlow.
- Frameworki uczenia maszynowego:
- PyTorch – elastyczny framework umożliwiający szybkie prototypowanie oraz trenowanie sieci neuronowych.
- TensorFlow – kompleksowe narzędzie do budowy, trenowania i wdrażania modeli uczenia maszynowego.
- Rotacyjnie inwariantne architektury sieci neuronowych – analiza nowych rozwiązań badanych w literaturze naukowej, takich jak ta opisana w publikacji „General E(2)-Equivariant Steerable CNNs”.
- Wykorzystanie akceleracji GPU (NVIDIA) - obliczenia zostały znacząco przyspieszone dzięki użyciu kart graficznych NVIDIA GeForce RTX 3060 12GB, które zapewniają wydajne środowisko dla operacji obliczeniowo intensywnych. Frameworki takie jak PyTorch i TensorFlow wspierają technologie CUDA oraz Tensor, umożliwiając efektywne wykorzystanie zasobów GPU.
- Konteneryzacja za pomocą Dockera – narzędzie do tworzenia odizolowanych środowisk uruchomieniowych, które zapewniło łatwość replikacji środowiska oraz współdzielenia projektu, także w konfiguracjach z obsługą GPU.

Zakres tematyczny

Organizacja pracy

Podstawy teoretyczne

Wprowadzenie do sieci konwolucyjnych (CNN)

Inwariancja translacyjna i rotacyjna

Problemy z rotacyjną inwariancją w klasycznych CNN

Przegląd literatury (np. E(2)-Equivariant CNNs, CyCNN)

Opis zbiorów danych

MNIST (cyfry odręczne)

GTSRB Gray (znaki drogowe w odcieniach szarości)

GTSRB RGB (znaki drogowe)

LEGO (obiekty 3d rzutowane na 2d)

Sposób augmentacji danych: zakresy rotacji, łączenie zbiorów

Architektury modeli

Standardowe CNN

Rotacyjnie inwariantne sieci (np. CyResNet, CyVGG, G-CNN)

Transformacje polarne: linearpolar vs logpolar

Implementacja i środowisko eksperymentalne

Python, PyTorch

Struktura projektu

Automatyzacja: skrypty trenowania, testowania, ewaluacji

Obsługa GPU, Docker, WSL

Organizacja logów, modeli, confusion matrixów

Eksperymenty

Scenariusze trenowania/testowania (opis JSON)

Pomiar skuteczności (accuracy, macierze pomyłek)

Śledzenie metryk: średnia, mediana, odchylenie standardowe

Analiza skuteczności względem rotacji

Ranking modeli

Porównanie wyników

CyCNN vs klasyczne CNN

VGG vs CyVGG

Resnet vs CyResNet

CyVGG vs CyResNet

[...] cyresnet56 uczył się dłużej niż cyvgg19, ale dawał bardziej stabilne wyniki, w szczególności w przypadku funkcji aktywacji typu logpolar. I jak to się mówi – nie można zjeść ciastka i mieć go też²¹.

Wpływ transformacji (linearpolar vs logpolar)

Wydajność na różnych zbiorach

Wnioski

Skuteczność rotacyjnych architektur

Wnioski z automatyzacji i systematyzacji ewaluacji

Propozycje dalszych badań

Bibliografia

Aneks

Listingi kodów

Dodatkowe wykresy, tablice wyników